INTRODUCTION TO OOPS AND JAVA

Module 1

2nd Semester MCA

The Genesis of Java

- • Originated in 1991 as part of the Green Project by Sun Microsystems.
- • Initially designed for embedded systems; originally named 'Oak'.
- • Renamed to Java in 1995 due to trademark issues.
- • Designed to be platform-independent and secure for networked devices.

Key Features of Java

- • Simple and familiar syntax similar to C/C++.
- • Object-Oriented Programming (OOP) principles.
- • Platform-independent through JVM (Write Once, Run Anywhere).
- • Robust and Secure.
- • Multithreaded and High Performance (JIT Compiler).
- • Distributed and Dynamic.

JVM, JRE, and JDK

- • JVM (Java Virtual Machine): Executes Java bytecode.
- • JRE (Java Runtime Environment): JVM + core libraries.
- • JDK (Java Development Kit): JRE + compiler + debugging tools.
- Together they form the Java platform.

Java Compilation and Execution Process

- 1. Java source code (.java) → compiled by javac → Bytecode (.class)
- 2. Bytecode → interpreted/compiled by JVM (JIT).
- 3. Output is platform-independent and portable.

Java Version Evolution (JDK 1.0 → Java 17)

- • JDK 1.0 (1996): Initial release
- • J2SE 1.2 (1998): Swing, Collections
- • J2SE 5 (2004): Generics, Enhanced for-loop
- • Java SE 8 (2014): Lambda, Streams
- • Java SE 9 (2017): Modules
- • Java SE 14–17: Records, Sealed classes, Switch expressions

APPLICATION INTERFACE

The *application program interface (API)*, also known as *library*, contains predefined classes and interfaces for developing Java programs

The *Java Development Toolkit (JDK)* consists of a set of separate programs, each invoked from a command line, for developing and testing Java programs. Instead of using the JDK, you can use a Java development tool (e.g., NetBeans, Eclipse, and TextPad)—software that provides an *integrated development environment (IDE)* for developing Java programs quickly

THE JAVA CLASS LIBRARIES (JCL) ESSENTIALS

The Java Class Libraries (JCL) are a **collection of dynamic libraries** that make up a large part of the **Java Development Kit (JDK)** and the **Java Runtime Environment (JRE)**. They provide pre-written, reusable code for common tasks, forming the core of the Java platform.

Category	Description	Example Packages
Java Platform API (The Standard Edition - Java SE)	The essential and fundamental classes.	java.lang, java.io, java.util, java.net
Optional/Extension APIs (Used for specialized tasks)	Libraries for advanced functionality like databases or graphics.	javax.sql, javax.swing, java.rmi

JDK VERSIONS

JDK 1.02 (1995)

JDK 1.1 (1996)

JDK 1.2 (1998)

JDK 1.3 (2000)

JDK 1.4 (2002)

JDK 1.5 (2004) a. k. a. JDK 5 or Java 5

JDK 1.6 (2006) a. k. a. JDK 6 or Java 6

JDK 1.7 (2011) a. k. a. JDK 7 or Java 7

JDK 1.8 (2014) a. k. a. JDK 8 or Java 8

JDK EDITIONS

Java Standard Edition (J2SE)

- J2SE can be used to develop client-side standalone applications or applets.

Java Enterprise Edition (J2EE)

- J2EE can be used to develop server-side applications such as Java servlets, Java ServerPages, and Java ServerFaces.

Java Micro Edition (J2ME).

- J2ME can be used to develop applications for mobile devices such as cell phones.

This book uses J2SE to introduce Java programming.

Key Characteristics of Java

- Object-oriented and modular
- Architecture-neutral and portable
- Secure execution environment
- Robust memory management with garbage collection

Object-Oriented Programming (OOP)

- Programming approach based on objects and classes
- Combines data and methods into a single unit
- Improves code reusability and maintainability
- Java strictly follows the OOP paradigm

Two Programming Paradigms

- Procedural Programming: focuses on functions and procedures
- Object-Oriented Programming: focuses on objects
- Java primarily supports object-oriented programming
- Procedural concepts are used only in limited form

Abstraction

- Abstraction means showing only essential features
- Implementation details are hidden from the user
- Achieved using classes, interfaces, and abstract methods
- Helps reduce complexity and improve design clarity

The Three OOP Principles

- Encapsulation: binding data and methods together
- Inheritance: acquiring properties of existing classes
- Polymorphism: ability to take many forms

Encapsulation

- Data is hidden using access modifiers
- Prevents direct access to internal data
- Improves security and maintainability
- Implemented using private variables and public methods

Inheritance

- Allows one class to inherit properties of another
- Promotes code reuse
- Uses 'extends' keyword in Java
- Supports hierarchical relationships

Polymorphism

- One interface, multiple implementations
- Achieved through method overloading and overriding
- Enhances flexibility and extensibility
- Important for dynamic method dispatch

A SIMPLE JAVA PROGRAM

Every Java program must have at least one class. Each class has a name. By convention, class names start with an uppercase letter.

In a program, multiple classes may cooperate with each other.

Each program needs an entry point to start execution.

The program is executed from the **main** method that is defined in one of the classes. A class may contain several methods. The **main** method is the entry point where the program begins execution.

Example: Hello Java

```
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

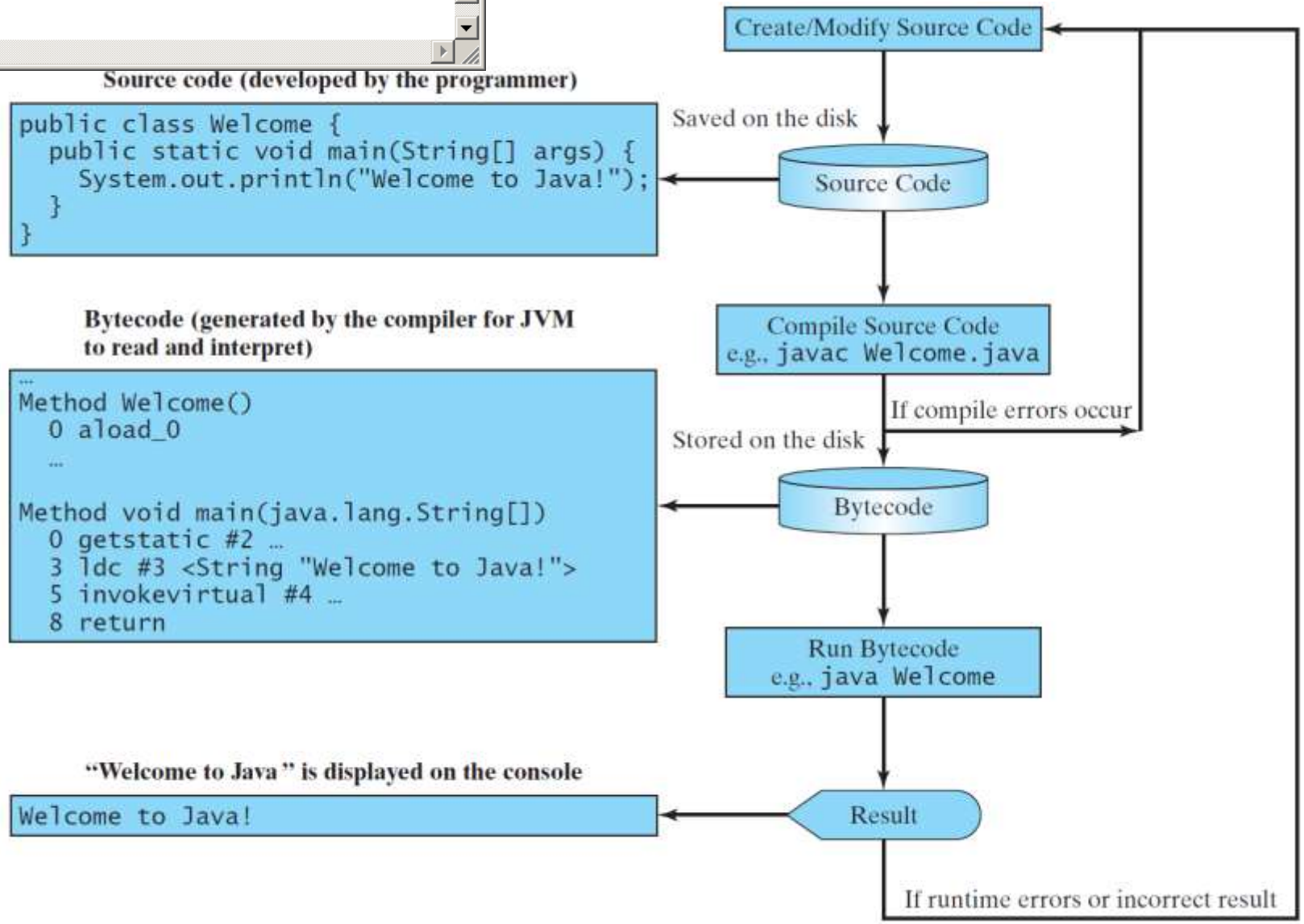
- Output: Hello, Java!

CREATING, COMPILING, AND RUNNING PROGRAMS

Welcome - Notepad

File Edit Format View Help

```
// This application program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```



COMPILING JAVA SOURCE CODE

The Java language is a high-level language, but Java bytecode is a low-level language. The *bytecode* is similar to machine instructions but is architecture neutral and can run on any platform that has a *Java Virtual Machine (JVM)*.

The virtual machine is a program that interprets Java bytecode. This is one of Java's primary advantages: *Java bytecode can run on a variety of hardware platforms and operating systems.*

Java source code is compiled into Java bytecode (.class file) and Java bytecode is interpreted by the JVM. Your Java code may use the code in the Java library. To execute a Java program is to run the program's bytecode.

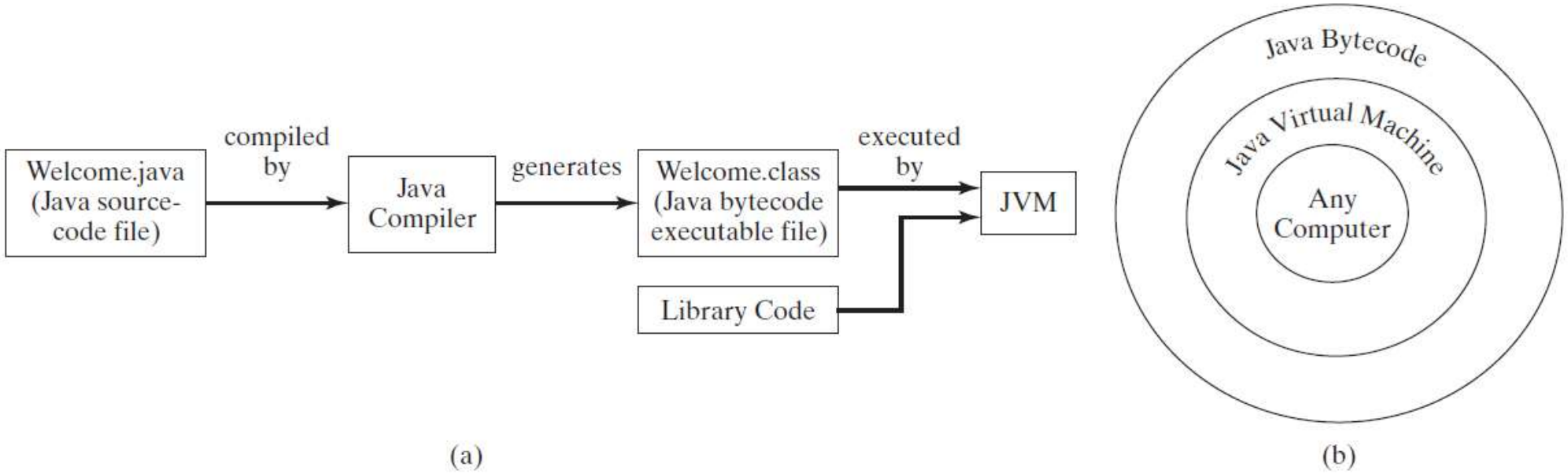
COMPILING JAVA SOURCE CODE

You can execute the bytecode on any platform with a JVM, which is an interpreter and resides in RAM. It translates the individual instructions in the bytecode into the target machine (i.e. the machine on which it is working at) language code one at a time rather than the whole program as a single unit.

Each step is executed immediately after it is translated.

The bytecode can then run on any computer with a Java Virtual Machine, as shown below. In fact, Java Virtual Machine is a software that interprets Java bytecode.

COMPILING JAVA SOURCE CODE



PROCEDURAL VERSUS OBJECT-ORIENTED PROGRAMMING

- The procedural programming aims at solving problems using a collection of variables and functions. This approach is not manageable as the size of the code gets large.
- Object-oriented programming (OOP) is an alternative approach where variables and functions are combined to form classes.
- The procedural paradigm focuses on designing methods. The object-oriented paradigm couples data and methods together into objects. Software design using the object-oriented paradigm focuses on objects and operations on objects.
- Classes provide more flexibility and modularity for building reusable software.

TRACE A PROGRAM EXECUTION

Enter main method

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

TRACE A PROGRAM EXECUTION

Execute statement

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

TRACE A PROGRAM EXECUTION

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

print a message to the console

ANATOMY OF A JAVA PROGRAM

- Class name
- Main method
- Statements
- Statement terminator
- Reserved words
- Comments
- Blocks

CLASS NAME

Every Java program must have at least one class. Each class has a name. By convention, class names start with an uppercase letter. In this example, the class name is Welcome.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

MAIN METHOD

Line 2 defines the main method. In order to run a class, the class must contain a method named main. The program is executed from the main method.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

STATEMENT

A statement represents an action or a sequence of actions. The statement System.out.println("Welcome to Java!") in the program below is a statement to display the greeting "Welcome to Java!”.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

STATEMENT TERMINATOR

Every statement in Java ends with a semicolon (;).

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

RESERVED WORDS

Reserved words or keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program. For example, when the compiler sees the word **class**, it understands that the word after class is the name for the class.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

BLOCKS

A pair of braces in a program forms a block that groups components of a program.

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Class block

Method block

SPECIAL SYMBOLS

Character	Name	Description
{}	Opening and closing braces	Denotes a block to enclose statements.
()	Opening and closing parentheses	Used with methods.
[]	Opening and closing brackets	Denotes an array.
//	Double slashes	Precedes a comment line.
" "	Opening and closing quotation marks	Enclosing a string (i.e., sequence of characters).
;	Semicolon	Marks the end of a statement.

{ … }

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

(...)

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

;

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

// ...

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

" "
...

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

PROGRAMMING STYLE AND DOCUMENTATION

Appropriate Comments

Naming Conventions

Proper Indentation and Spacing Lines

Block Styles

APPROPRIATE COMMENTS

Include a summary at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques it uses.

In addition to line comments (beginning with //) and block comments (beginning with /*), Java supports comments of a special type, referred to as *javadoc comment* (begin with /** and end with */). They can be extracted into an HTML file using the JDK's **javadoc** command. Use javadoc comments (/** ... */) for commenting on an entire class or an entire method.

NAMING CONVENTIONS

Choose meaningful and descriptive names.

Class names:

- Capitalize the first letter of each word in the name. For example, the class name `ComputeExpression`.

PROPER INDENTATION AND SPACING

A consistent indentation style makes programs clear and easy to read, debug, and maintain. *Indentation* is used to illustrate the structural relationships between a program's components or statements.

Spacing

- Use blank line to separate segments of the code.

BLOCK STYLES

There are two popular styles, *next-line* style and *end-of-line* style, as shown below.

Next-line style

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

End-of-line style

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```

JAVA DATA TYPES

JAVA: A STRONGLY-TYPED LANGUAGE

Java is **strongly-typed**, meaning that **every variable must have a declared type**, and that type is enforced strictly by the compiler and at runtime.

Explicit Type Declaration: You must explicitly state the data type for every variable when it is declared (e.g., int count = 10;, not just count = 10;).

Compile-Time Checking (Safety First!):

- The Java compiler checks for type compatibility before the program is run.
- This prevents many errors (e.g., trying to assign a String to an int) that would otherwise appear only at runtime in dynamically-typed languages.
- This leads to more robust and easier-to-debug code.

No Implicit Conversions: Java generally does not perform implicit data type conversion between incompatible type.

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

Primitive data types: The primitive data types include boolean, char, byte, short, int, long, float and double.

Non-primitive data types: The non-primitive data types include Classes, Interfaces, and Arrays.

JAVA PRIMITIVE DATA TYPES

There are 8 types of primitive data types:

Boolean data type

byte data type

char data type

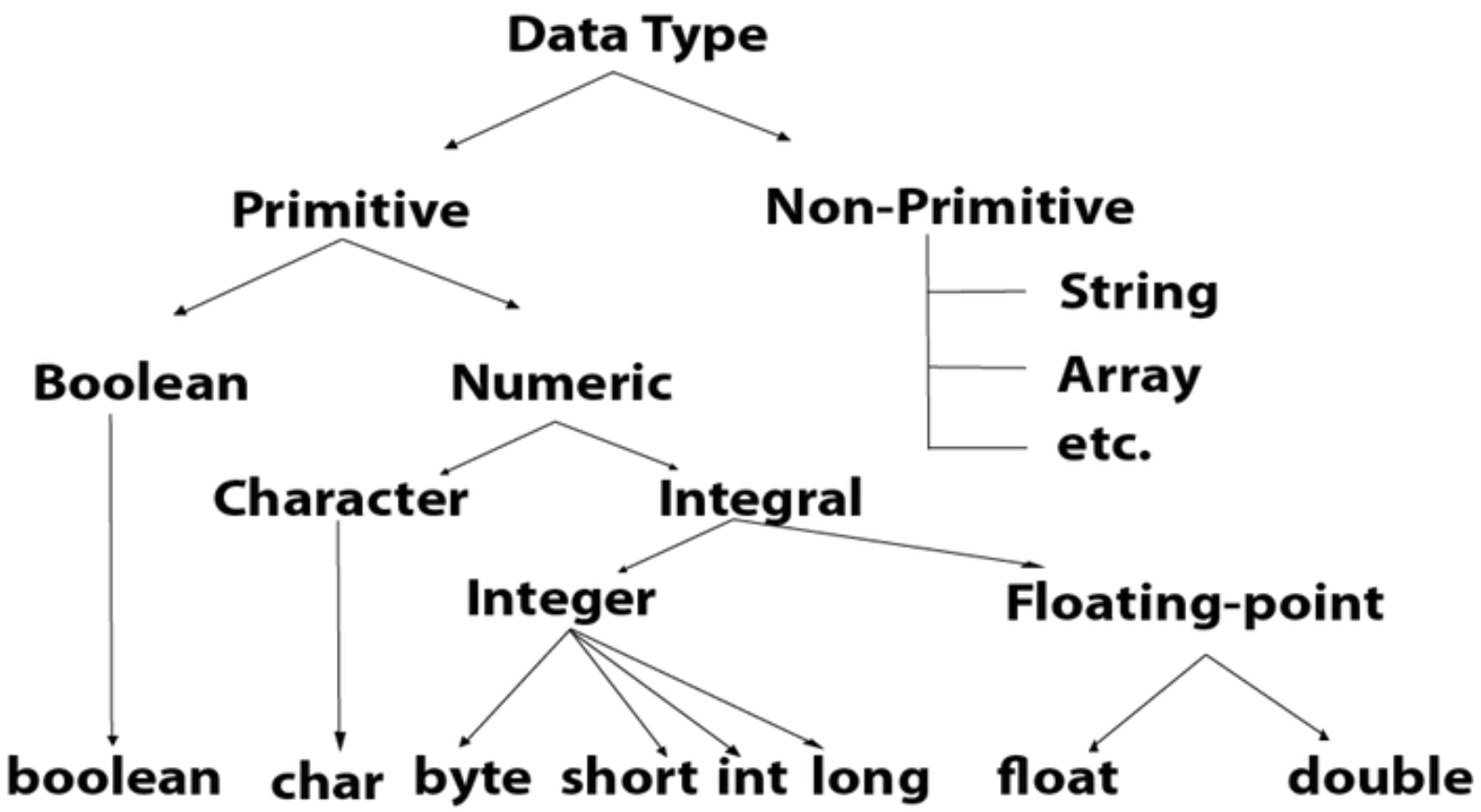
short data type

int data type

long data type

float data type

double data type


Data Type
Primitive
Non-Primitive
String
Array
etc.
Boolean
Numeric
Character
Integral
Integer
Floating-point
boolean
char
byte
short
int
long
float
double

BOOLEAN DATA TYPE

The Boolean data type is used to store only two possible values: true and false.

This data type is used for simple flags that track true/false conditions.

The Boolean data type specifies one bit of information.

Example: Boolean one = false

BYTE DATA TYPE

The byte data type is an example of primitive data type.

It is an 8-bit signed two's complement integer.

Its value-range lies between -128 to 127 (inclusive).

Its default value is 0.

The byte data type is used to save memory in large arrays where the memory savings is most required.

It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.

Example: byte a = 10, byte b = -20

SHORT DATA TYPE

The short data type is a 16-bit signed two's complement integer.

Its value-range lies between -32,768 to 32,767 (inclusive).

Its default value is 0.

The short data type can also be used to save memory just like byte data type.

A short data type is 2 times smaller than an integer.

Example: short s = 10000, short r = -5000

INT DATA TYPE

The int data type is a 32-bit signed two's complement integer.

Its value-range lies between - 2,147,483,648 (-2^31) to 2,147,483,647 (2^31 -1) (inclusive).

Its default value is 0.

The int data type is generally used as a default data type for integral values unless if there is no problem about memory.

Example: int a = 100000, int b = -200000

LONG DATA TYPE

The long data type is a 64-bit two's complement integer.

Its value-range lies between -9,223,372,036,854,775,808(-2^63) to 9,223,372,036,854,775,807(2^63 -1)(inclusive).

Its default value is 0.

The long data type is used when you need a range of values more than those provided by int.

Example: long a = 100000L, long b = -200000L

FLOAT DATA TYPE

The float data type is a single-precision 32-bit IEEE 754 floating point.

It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers.

The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Example: float f1 = 234.5f

DOUBLE DATA TYPE

The double data type is a double-precision 64-bit IEEE 754 floating point.

Its value range is unlimited.

The double data type is generally used for decimal values just like float.

The double data type also should never be used for precise values, such as currency.
Its default value is 0.0d.

Example: double d1 = 12.3

CHAR DATA TYPE

The char data type is a single 16-bit Unicode character.

Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive).

The char data type is used to store characters.

Example: char letterA = 'A'

Why char uses 2 byte in java and what is \u0000 ?

It is because java uses Unicode system not ASCII code system. The \u0000 is the lowest range of Unicode system.

UNICODE SYSTEM

Unicode is a universal international standard character encoding that is capable of representing most of the world's written languages.

In unicode, character holds 2 byte, so java also uses 2 byte for characters.

lowest value:\u0000(\u is unicode escape)

highest value:\uFFFF

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

JAVA LITERALS: FIXED VALUES IN CODE

A **literal** is a source code representation of a fixed, constant value. It's the raw data we assign to variables.

Literals are values written directly into the code (e.g., the number *5*, the character 'A').

Java assumes a **default data type** for numeric literals:

- Whole numbers (e.g., 100) are treated as *int*.
- Decimal numbers (e.g., 3.14) are treated as *double*.

Suffixes for Specificity: You must use suffixes to change the default type:

- Use L (or l) for a long integer literal (e.g., 5000000000L).
- Use F (or f) for a float decimal literal (e.g., 1.5F).

Enclosures Define Type: Single quotes (' ') denote a **char** literal ('C'), while double quotes (" ") denote a **String** literal ("Code").

EXAMPLES FOR LITERALS

Code Statement	Literal Type	Data Type Assigned
int num = 42;	Integer Literal	int
double pi = 3.14159;	Floating-Point Literal	double
float rate = 0.5f;	Floating-Point Literal	float (Explicitly cast)
char grade = 'A';	Character Literal	char
boolean flag = true;	Boolean Literal	boolean

JAVA VARIABLES

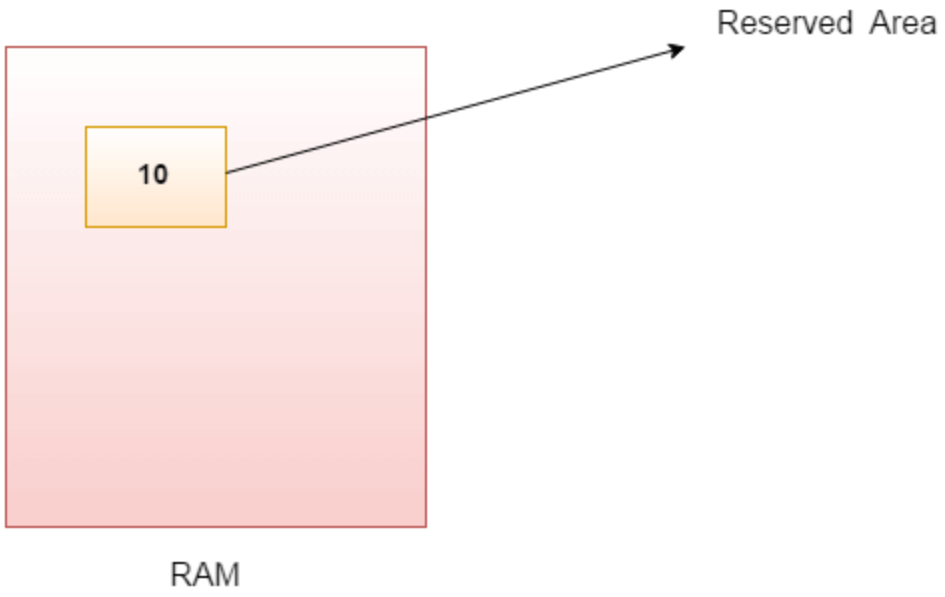
A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.

VARIABLE

Variable is name of *reserved area allocated in memory*.

In other words, it is a *name of memory location*. It is a combination of "vary + able" that means its value can be changed.



1) LOCAL VARIABLE

A variable declared inside the body of the method is called local variable.

You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) INSTANCE VARIABLE

A variable declared inside the class but outside the body of the method, is called instance variable. It is not declared as static.

It is called instance variable because its value is instance specific and is not shared among instances.

3) STATIC VARIABLE

A variable which is declared as static is called static variable. It cannot be local

Memory allocation for static variable happens only once when the class is loaded in the memory.

EXAMPLE

```
class A{
int data=50;//instance variable
static int m=100;//static variable
void method(){
        int n=90;//local variable
}
}//end of class
```

TYPE INFERENCE WITH LOCAL VARIABLES

Type inference for local variables in Java, introduced in **Java 10**, allows you to use the keyword **var** instead of explicitly stating the data type of a local variable when its type can be deduced by the compiler from the initializer expression.

It's a reserved type name used as a placeholder.

The compiler infers the actual type at compile time. Once the type is inferred, the variable is **statically typed** and its type **cannot change**.

Example: Instead of string name = “java”;, var name = “java”;

AUTOMATIC TYPE PROMOTION

Automatic Type Promotion (also called Widening Conversion) is Java's mechanism for converting a value from a data type of smaller range to a data type of a larger range **without requiring an explicit cast.**

Safety First: The primary rule is that the destination type must be able to hold the entire range of the source type. This conversion is **non-lossy**.

Conversion Path: When promoting, types follow a specific hierarchy from smallest to largest:

$$\text{byte} \rightarrow \text{short} \rightarrow \text{int} \rightarrow \text{long} \rightarrow \text{float} \rightarrow \text{double}$$

Promotion frequently happens automatically in mixed-type expressions.

JAVA VARIABLE EXAMPLE: WIDENING

```
class Simple{
public static void main(String[] args){
        int a=10;
        float f=a;
        System.out.println(a);
        System.out.println(f);
}
}
```

Output:
10
10.0

JAVA VARIABLE EXAMPLE: NARROWING (TYPECASTING)

```
class Simple{
public static void main(String[] args){
        float f=10.5f;
        //int a=f;//Compile time error
        int a=(int)f;
        System.out.println(f);
        System.out.println(a);
}
}
```

Output:
10.5
10

JAVA ARRAYS

An array is a collection of similar type of elements which has contiguous memory location.

We can store only a fixed set of elements in a Java array.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

Syntax to Declare an Array in Java

- dataType[] arr; (or)
- dataType []arr; (or)
- dataType arr[];

Instantiation of an Array in Java

- arrayRefVar=new datatype[size];

```
class Testarray{
public static void main(String args[]){
        int a[]=new int[5];//declaration and instantiation
        a[0]=10;//initialization
        a[1]=20     ;
        a[2]=70;
        a[3]=40;
        a[4]=50     ; // int a[]={10,20,70,40,50};
        //traversing array
        for(int i=0;i<a.length;i++)//length is the property of array
                System.out.println(a[i]);
}
}
```

MULTI-DIMENSIONAL ARRAYS IN JAVA

A **multi-dimensional array** is essentially an array of arrays. It's used to store data in a table-like format (rows and columns) or a grid structure.

In Java, a multi-dimensional array is primarily implemented as a two-dimensional array (array of array), representing a matrix or table.

Declaration: dataType[][] arrayName;

Instantiation arrayName = new dataType[rows][columns];

RETURNING AN ARRAY FROM A METHOD

```
public static int[] reverse(int[] list) {
        int[] result = new int[list.length];
        for (int i = 0, j = result.length - 1; i < list.length; i++, j--) {
                result[j] = list[i];
        }
  return result;
}
```

OPERATORS IN JAVA

Operator in Java is a symbol which is used to perform operations. For example: +, -, *, / etc.

There are many types of operators in Java which are given below:

- Unary Operator,
- Arithmetic Operator,
- Shift Operator,
- Relational Operator,
- Bitwise Operator,
- Logical Operator,
- Ternary Operator and
- Assignment Operator.

JAVA UNARY OPERATOR

The Java unary operators require only one operand. Unary operators are used to perform various operations i.e.:

incrementing/decrementing a value by one

negating an expression

inverting the value of a boolean

Java Unary Operator Example: ++

```
class OperatorExample{
public static void main(String args[]){
        int x=10;
        System.out.println(x++);//10 (11)
        System.out.println(++x);//12
        System.out.println(x--);//12 (11)
        System.out.println(--x);//10
}
}
```

```
class OperatorExample{
public static void main(String args[]){
        int a=10;
        int b=10;
        System.out.println(a++ + ++a);          //10+12=22
        System.out.println(b++ + b++);          //10+11=21
}
}
```

JAVA ARITHMETIC OPERATORS

Java arithmetic operators are used to perform addition, subtraction, multiplication, and division. They act as basic mathematical operations.

EQUALITY AND RELATIONAL OPERATORS

Operator	Description	Example
`==`	equal to	5 == 3 is evaluated to `false`
`!=`	not equal to	5 != 3 is evaluated to `true`
`>`	greater than	5 > 3 is evaluated to `true`
`<`	less than	5 < 3 is evaluated to `false`
`>=`	greater than or equal to	5 >= 5 is evaluated to `true`
`<=`	less than or equal to	5 <= 5 is evaluated to `true`

```
class RelationalOperator {
public static void main(String[] args) {
        int number1 = 5, number2 = 6;
        if (number1 > number2) {
                System.out.println("number1 is greater than number2."); }
        else {
                System.out.println("number2 is greater than number1."); }
         }
 }
```

LOGICAL OPERATORS

Operator	Description	Example
`||`	conditional-OR: `true` if either of the boolean expression is `true`	`false || true` is evaluated to `true`
`&&`	conditional-AND: true if all boolean expressions are `true`	`false && true` is evaluated to `false`

```
class LogicalOperator {
    public static void main(String[] args) {
        int number1 = 1, number2 = 2, number3 = 9;
        boolean result;
        // At least one expression needs to be true for the result to be true
        result = (number1 > number2) || (number3 > number1);
        // result will be true because (number1 > number2) is true
        System.out.println(result);
        // All expression must be true from result to be true
        result = (number1 > number2) && (number3 > number1);
        // result will be false because (number3 > number1) is false
        System.out.println(result);
    }
}
```

BITWISE AND BIT SHIFT OPERATORS

Operator	Description
~	Bitwise Complement
<<	Left Shift
>>	Right Shift
>>>	Unsigned Right Shift
&	Bitwise AND
^	Bitwise exclusive OR
|	Bitwise inclusive OR

JAVA LEFT SHIFT OPERATOR

The Java left shift operator << is used to shift all of the bits in a value to the left side of a specified number of times.

```
class OperatorExample{
  public static void main(String args[]){
System.out.println(10<<2);//10*2^2=10*4=40
System.out.println(10<<3);//10*2^3=10*8=80
System.out.println(20<<2);//20*2^2=20*4=80
System.out.println(15<<4);//15*2^4=15*16=240
}
}
```

JAVA RIGHT SHIFT OPERATOR

The Java right shift operator >> is used to move left operands value to right by the number of bits specified by the right operand.

```
class OperatorExample{
public static void main(String args[]){
System.out.println(10>>2);//10/2^2=10/4=2
System.out.println(20>>2);//20/2^2=20/4=5
System.out.println(20>>3);//20/2^3=20/8=2
}
}
```

JAVA AND OPERATOR EXAMPLE: LOGICAL && AND BITWISE &

The logical && operator doesn't check second condition if first condition is false. It checks second condition only if first one is true.

The bitwise & operator always checks both conditions whether first condition is true or false.

```
class OperatorExample{
public static void main(String args[]){
        int a=10;
        int b=5;
        int c=20;
        System.out.println(a<b&&a<c);
        //false && true = false
        System.out.println(a<b&a<c);
        //false & true = false
}
}
```

JAVA TERNARY OPERATOR

Java Ternary operator is used as one liner replacement for if-then-else statement and used a lot in Java programming. it is the only conditional operator which takes three operands.

```
class OperatorExample{
public static void main(String args[]){
        int a=2;
        int b=5;
        int min=(a<b)?a:b;
        System.out.println(min);
}
}
```

JAVA ASSIGNMENT OPERATOR

Java assignment operator is one of the most common operator. It is used to assign the value on its right to the operand on its left.

```
class OperatorExample{
public static void main(String args[]){
        int a=10;
        int b=20;
        a+=4;//a=a+4 (a=10+4)
        b-=4;//b=b-4 (b=20-4)
        System.out.println(a);
        System.out.println(b);
}
}
```

Operator Type	Category	Precedence
Unary	postfix	`expr++ expr--`
	prefix	`++expr --expr +expr -expr ~ !`
Arithmetic	multiplicative	`* / %`
	additive	`+ -`
Shift	shift	`<< >> >>>`
Relational	comparison	`< > <= >= instanceof`
	equality	`== !=`
Bitwise	bitwise AND	`&`
	bitwise exclusive OR	`^`
	bitwise inclusive OR	`|`
Logical	logical AND	`&&`
	logical OR	`||`
Ternary	ternary	`? :`
Assignment	assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`

OPERATOR PRECEDENCE IN JAVA

Operator Precedence determines the order in which operators in an expression are evaluated. It's similar to the **BODMAS/PEMDAS** rules in mathematics.

Higher Precedence Operators are evaluated **before** lower precedence operators.

Parentheses () can always be used to override the default precedence. Expressions inside parentheses are evaluated first.

Associativity determines the order of evaluation when multiple operators of the **same precedence** level appear consecutively.

Precedence Level	Operators	Associativity	Example of Operators
Highest	**Post-Unary**	Left-to-Right	[], ., (), ++ (postfix), -- (postfix)
	Pre-Unary	Right-to-Left	++ (prefix), -- (prefix), !, ~, +, -, new, cast (type)
	Multiplicative	Left-to-Right	*, /, %
	Additive	Left-to-Right	+, -
	Shift	Left-to-Right	<<, >>, >>>
	Relational	Left-to-Right	>, <, >=, <=, instanceof
	Equality	Left-to-Right	==, !=
	Bitwise AND	Left-to-Right	&[13]
	Bitwise XOR	Left-to-Right	^[16]
	Bitwise OR	Left-to-Right	|[19]
	Logical AND	Left-to-Right	&&
	Logical OR	Left-to-Right	||
	Ternary	Right-to-Left	? :
Lowest	**Assignment**	Right-to-Left	=, +=, -=, *=, etc.

CONTROL STATEMENTS

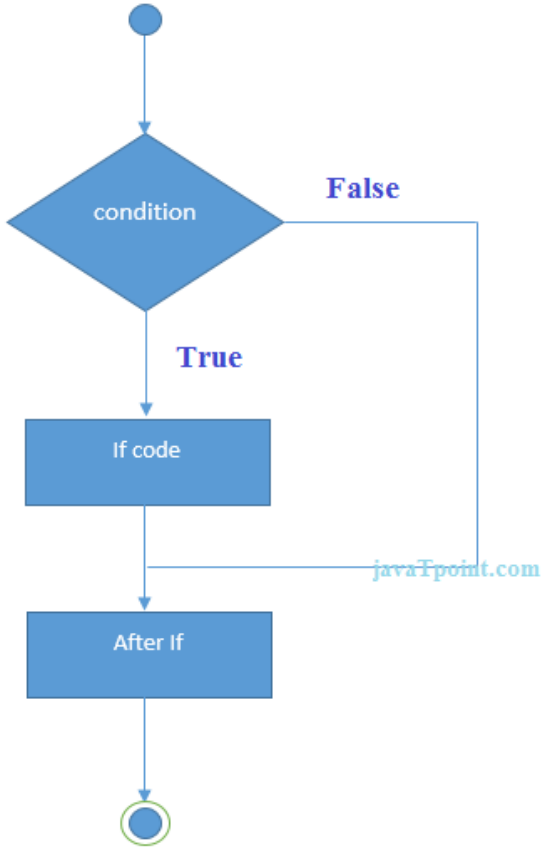
1. if statement
2. if-else statement
3. if-else-if ladder
4. nested if statement
5. switch

JAVA IF STATEMENT

The Java if statement tests the condition. It executes the *if block* if condition is true.

Syntax:

```
if(condition){
//code to be executed
}
```



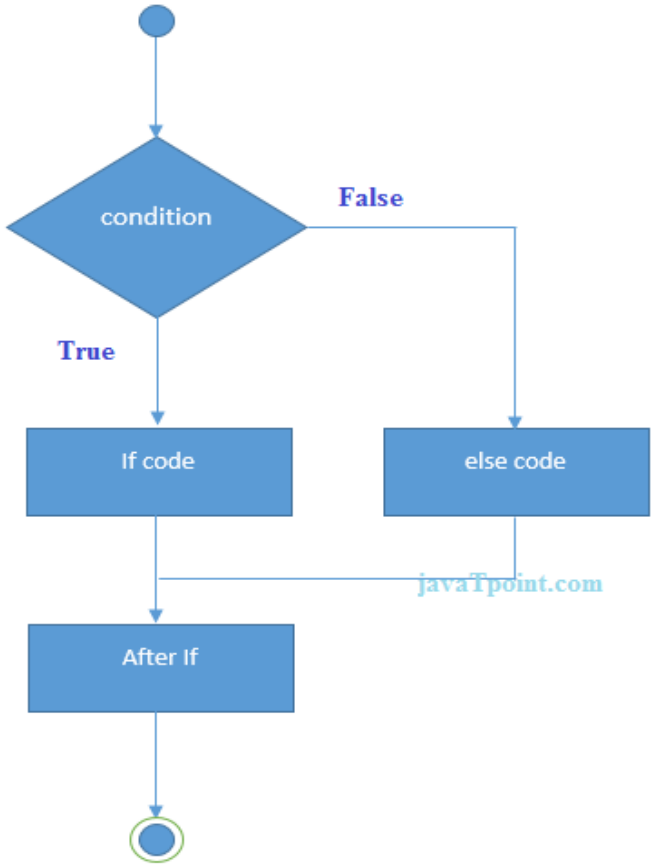
```
/Java Program to demonstrate the use of if statement.
public class IfExample {
public static void main(String[] args) {
            //defining an 'age' variable
               int age=20;
               //checking the age
               if(age>18){
                  System.o     ut.print("Age is greater than 18");
                        }
}
}
```

JAVA IF-ELSE STATEMENT

The Java if-else statement also tests the condition. It executes the *if block* if condition is true otherwise *else block* is executed.

Syntax:

```
if(condition){
//code if condition is true
}else{
//code if condition is false
}
```



NUM ODD OR EVEN

```
public class IfElseExample {
public static void main(String[] args) {
        //defining a variable
        int number=13;
        //Check if the number is divisible by 2 or not
        if(number%2==0){
            System.out.println("even number");
        }else{
            System.out.println("odd number");
                }
}
}
```

JAVA IF-ELSE-IF LADDER STATEMENT

The if-else-if ladder statement executes one condition from multiple statements.

Syntax:

```
if(condition1){
//code to be executed if condition1 is true
}else if(condition2){
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

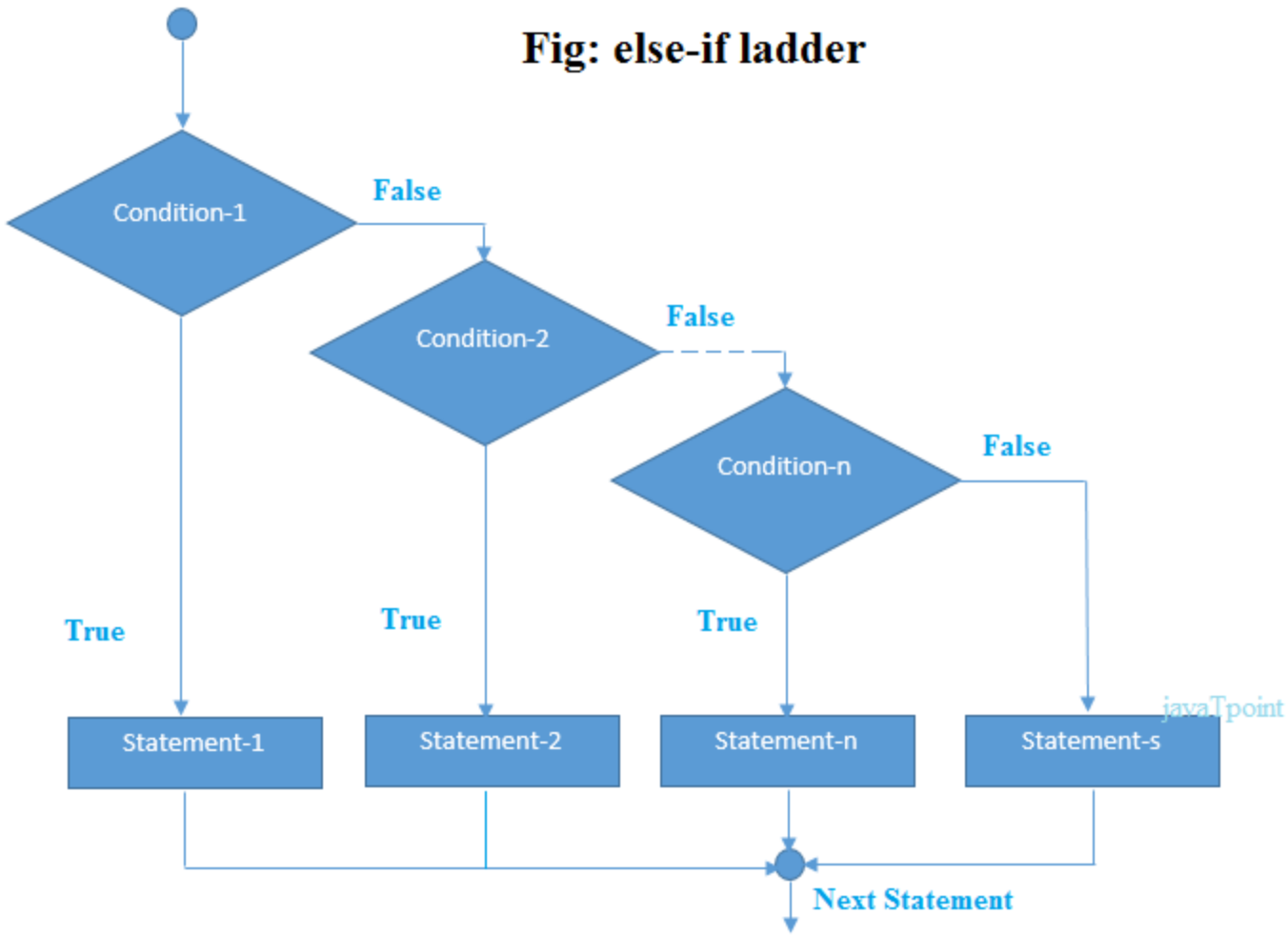


Fig: else-if ladder

CHECK NUM POSITIVE NEGATIVE OR ZERO

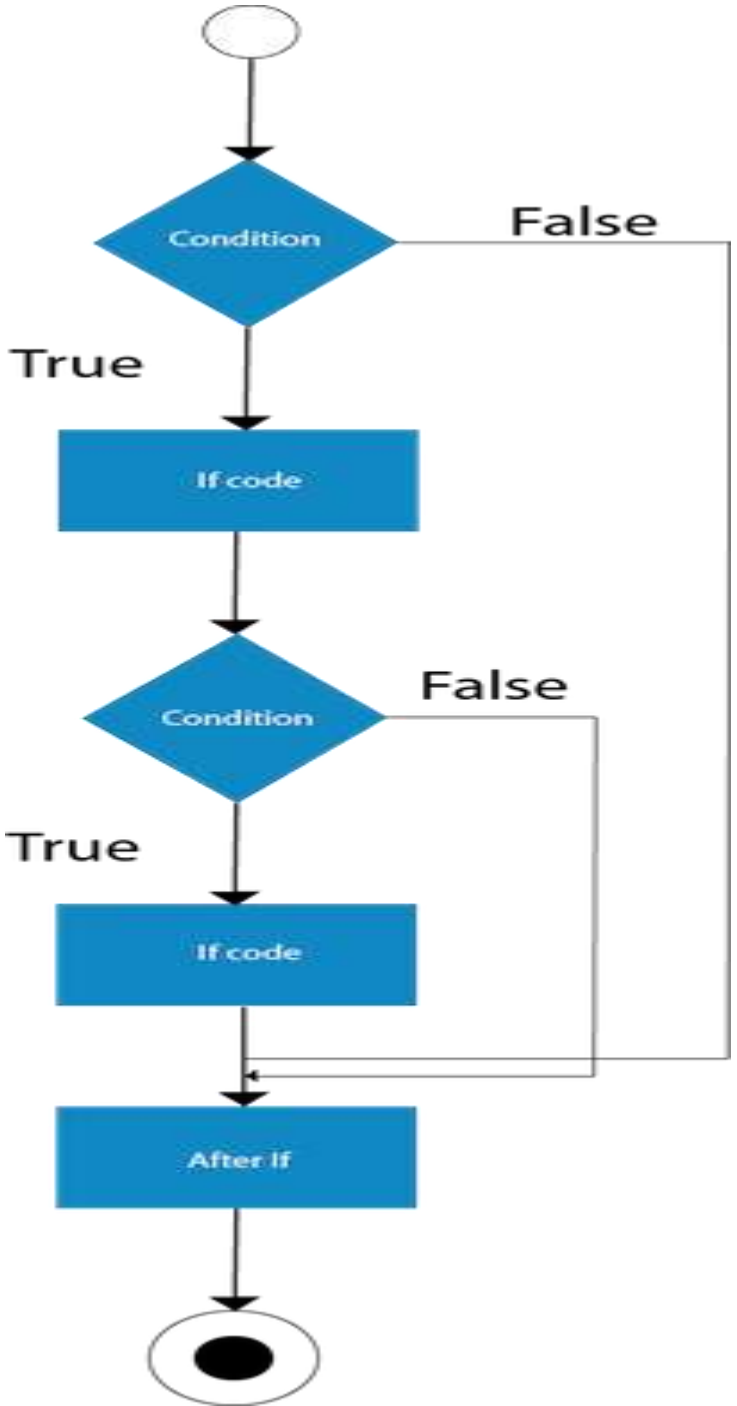
```
public class PositiveNegativeExample {
        public static void main(String[] args) {
    int number=-13;
    if(number>0){
            System.out.println("POSITIVE");
    }else if(number<0){
            System.out.println("NEGATIVE");
    }else{
            System.out.println("ZERO");
   }
        }
  }
```

JAVA NESTED IF STATEMENT

The nested if statement represents the *if block within another if block*. Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
if(condition){
    //code to be executed
        if(condition){
            //code to be executed
    }
}
```


Condition
False
True
If code
Condition
False
True
If code
After If

LARGEST AMONG 3 NUMBERS

```
public class Largest {

   public static void main(String[] args) {

            double n1 = -4.5, n2 = 3.9, n3 = 5.5;

            if(n1 >= n2) {

                 if(n1 >= n3)

                      System.out.println(n1 + " is the largest number.");

                 else

                      System.out.println(n3 + " is the largest number.");

            } else {

                  if(n2 >= n3)

            System.out.println(n2 + " is the largest number.");

        else

                 System.out.println(n3 + " is the largest number.");

                 }

   }
```

JAVA SWITCH STATEMENT

The Java *switch statement* executes one statement from multiple conditions.

Points to Remember

There can be *one or N number of case values* for a switch expression.

The case value must be of switch expression type only. The case value must be *literal or constant*. It doesn't allow variables.

The case values must be *unique*. In case of duplicate value, it renders compile-time error.

The Java switch expression must be of *byte, short, int, long (with its Wrapper type), and string*.

Each case statement can have a *break statement* which is optional. When control reaches to the break statement, it jumps the control after the switch expression. If a break statement is not found, it executes the next case.

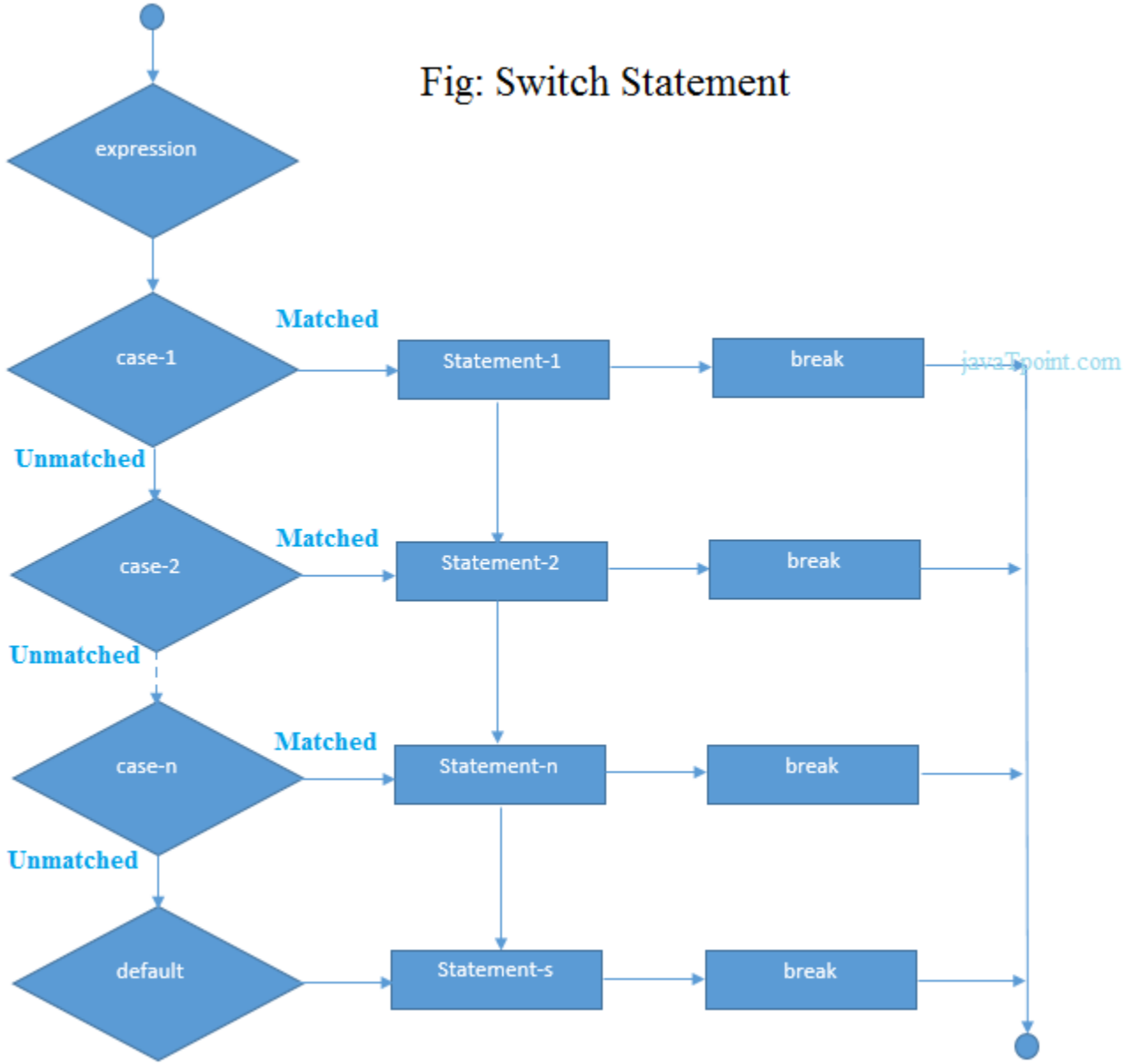
The case value can have a *default label* which is optional.

SYNTAX

```
switch(expression){
   case value1:
    //code to be executed;
    break;  //optional
   case value2:
    //code to be executed;
    break;  //optional
   ......

   default:
    code to be executed if all cases are not matched;
   }
```

Fig: Switch Statement



```
public class SwitchExample {
public static void main(String[] args) {
    //Declaring a variable for switch expression
    int number=20;
    //Switch expression
    switch(number){
                //Case statements
                case 10: System.out.println("10");
                break;
                case 20: System.out.println("20");
                break;
                case 30: System.out.println("30");
                break;
                //Default case statement
                default:System.out.println("Not in 10, 20 or 30");
                        }
}
}
```

JAVA SWITCH STATEMENT IS FALL-THROUGH

The Java switch statement is fall-through. It means it executes all statements after the first match if a break statement is not present.

```
public class SwitchExample2 {
public static void main(String[] args) {
    int number=20;
    //switch expression with int value
    switch(number){
    //switch cases without break statements
    case 10: System.out.println("10");
    case 20: System.out.println("20");
    case 30: System.out.println("30");
    default:System.out.println("Not in 10, 20 or 30");
    }
}
}
```

Output:
20
30
Not in 10, 20 or 30

LOOPS IN JAVA

for loop

while loop

do-while loop

FOR LOOP

The Java *for loop* is used to iterate a part of the program several times. It consists of four parts:

Initialization: It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.

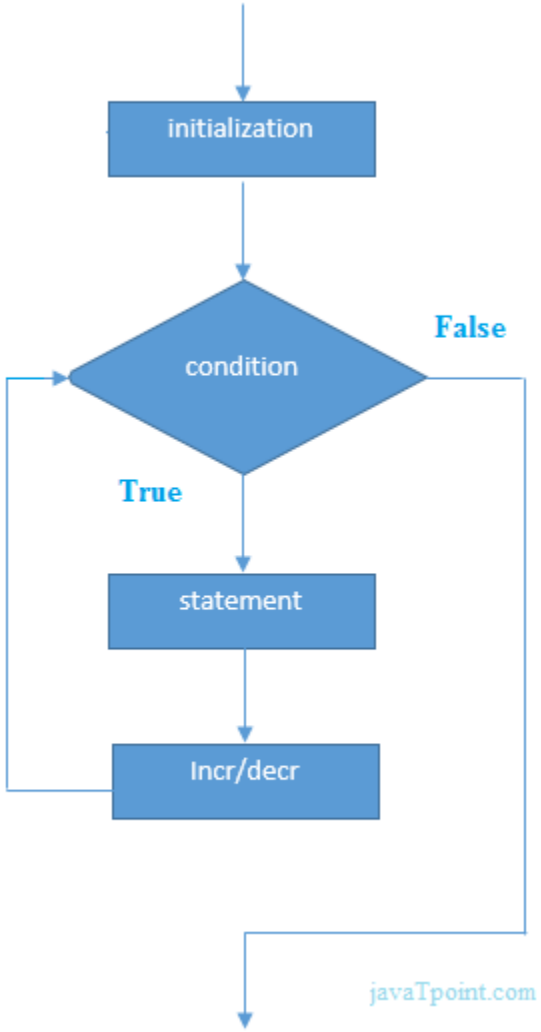
Condition: It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return boolean value either true or false. It is an optional condition.

Statement: The statement of the loop is executed each time until the second condition is false.

Increment/Decrement: It increments or decrements the variable value. It is an optional condition.

Syntax:

```
for(initialization;condition;incr/decr){
//statement or code to be executed
}
```



```
public class ForExample {
public static void main(String[] args) {
                    //Code of Java for loop
            for(int i=1;i<=10;i++){
                        System.out.println(i);
                    }
}
}
```

JAVA NESTED FOR LOOP

If we have a for loop inside another loop, it is known as nested for loop. The inner loop executes completely whenever outer loop executes.

```
public class PyramidExample {
public static void main(String[] args) {
        for(int i=1;i<=5;i++){
                for(int j=1;j<      =i;j++){
                        System.out.print("* ");
                }
                System.out.println();//new line
        }
}
}
```

Output:

```
*
*  *
*  *  *
*  *  *  *
*  *  *  *  *
```

JAVA FOR-EACH LOOP

The for-each loop is used to traverse array or collection in java. It is easier to use than simple for loop because we don't need to increment value and use subscript notation.

It works on elements basis not index. It returns element one by one in the defined variable.

Syntax:

```
for(Type var:array){

//code to be executed

}
```

```
//Java For-each loop example which prints the
//elements of the array
public class ForEachExample {
        public static void main(String[] args) {
    //Declaring an array
    int arr[]={12,23,44,56,78};
    //Printing array using for-each loop
    for(int i:arr){
            System.out.println(i);
        }
    }
}
```

JAVA INFINITIVE FOR LOOP

If you use two semicolons ;; in the for loop, it will be infinitive for loop.

Syntax:

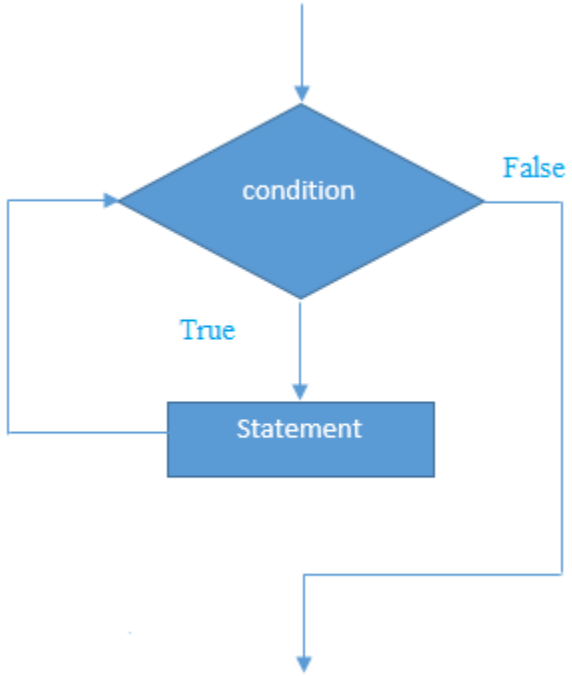
```
for(;;){
//code to be executed
}
```

JAVA WHILE LOOP

The Java *while loop* is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop.

Syntax:

```
while(condition){
//code to be executed
}
```



```
public class WhileExample {
public static void main(String[] args) {
                int i=1;
    while(i<=10){
                 System.out.println(i);
       i++;
                }
      }
}
```

JAVA INFINITIVE WHILE LOOP

If you pass **true** in the while loop, it will be infinitive while loop.
Syntax:

```
while(true){

//code to be executed

}
```

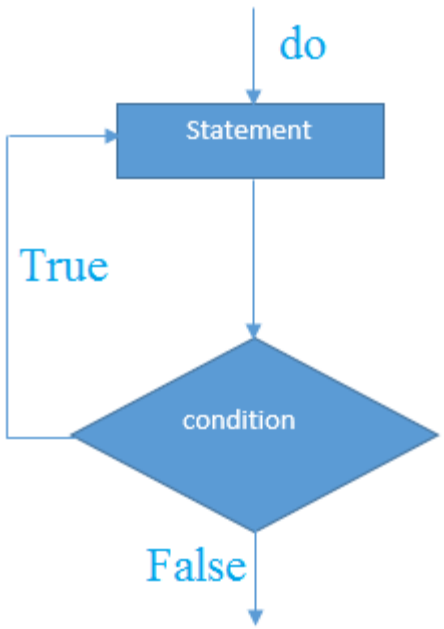
JAVA DO-WHILE LOOP

The Java *do-while loop* is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The Java *do-while loop* is executed at least once because condition is checked after loop body.

Syntax:

```
do{
//code to be executed
}while(condition);
```



```
public class DoWhileExample {
        public static void main(String[] args) {
                int i=1;
    do{
                  System.out.println(i);
                i++;
    }while(i<=10);
        }
  }
```

JAVA INFINITIVE DO-WHILE LOOP

If you pass **true** in the do-while loop, it will be infinitive do-while loop.

Syntax:

```
do{
//code to be executed
}while(true);
```

JAVA BREAK STATEMENT

When a break statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.

The Java *break* statement is used to break loop or switch statement. It breaks the current flow of the program at specified condition. In case of inner loop, it breaks only inner loop.

We can use Java break statement in all types of loops such as for loop, while loop and do-while loop.

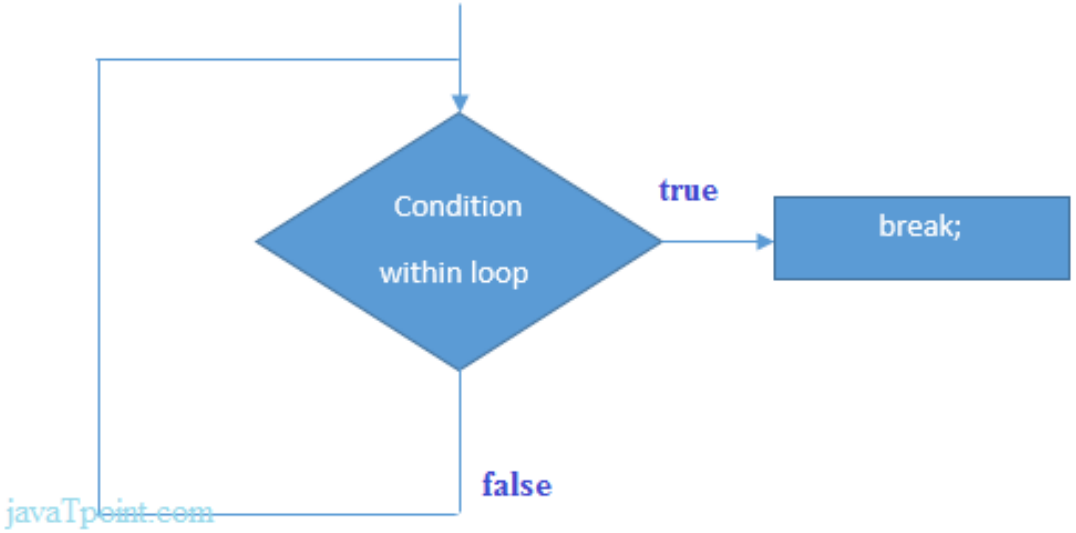


Figure: Flowchart of break statement

```
public class BreakExample {
public static void main(String[] args) {
    //using for loop
   for(int i=1;i<=10;i++){
      if(i==5){
                        //breaking the loop
         break;
                  }
      System.out.println(i);
               }
}
}
```

JAVA CONTINUE STATEMENT

The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately. It can be used with for loop or while loop.

The Java *continue statement* is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition. In case of an inner loop, it continues the inner loop only.

We can use Java continue statement in all types of loops such as for loop, while loop and do-while loop.

Syntax:

jump-statement;

continue;

```
public class ContinueExample {
public static void main(String[] args) {
    //for loop
    for(int i=1;i<=10;i++){
        if(i==5){
            //using continue statement
            continue;//it will skip the rest statement
        }
        System.out.println(i);
    }
}
}
```

PROGRAMMING ERRORS

Syntax Errors

- Detected by the compiler

Runtime Errors

- Causes the program to abort

Logic Errors

- Produces incorrect result

SYNTAX ERRORS

```
public class ShowSyntaxErrors {
  public static main(String[] args) {
    System.out.println("Welcome to Java);
  }
}
```

RUNTIME ERRORS

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    System.out.println(1 / 0);
  }
}
```

LOGIC ERRORS

```
public class ShowLogicErrors {
  public static void main(String[] args) {
    System.out.println("Celsius 35 is Fahrenheit degree ");
    System.out.println((9 / 5) * 35 + 32);
  }
}
```